ARIADNE · OVERNIGHT RUN

# What the relation slot was actually measuring

A pre-registered attempt to fix the rewrite stage's worst slot, the null result it produced, and the two measurement defects it uncovered on the way.

Night of 9–10 September 2026 · experiments EXP-2026-09-015 and EXP-2026-09-016 · generated 2026-09-10 00:23

### The short version

1. **The intended fix worked mechanically and changed nothing.** The relation slot can be read off the schema instead of guessed by the model, and it now is — but accuracy did not move, because the model was already picking a graph-consistent relation about as often.
2. **The metric was scoring the answer key, not the system.** Eleven of the 36 gold records that name a relation name one that does not exist between their own two labels. A system answering correctly was being marked wrong. This is the night's real finding.
3. **A second, independent measurement defect.** The arm that represents "let the AI write the query" silently answers nothing at its default settings, with no model call at all, while looking like a model that politely declined. Any cost figure taken from it would have been spectacularly wrong.
4. **A third finding, carried in and now verified.** Five kinds of connection in the CAPEC database are declared but not safety-approved, so the safety gate destroys correct queries before they run. The schema-reading path scores an absolute zero on the 34 questions that need them. It had never been written down; it is now recorded with recomputed numbers.
5. **The flags stay off.** Nothing shipped changed tonight.

## 1 · Why this was worth a night

The rewrite stage is the part of Ariadne that reads an analyst's question and decides *what is being asked* before anything touches a database. It fills five slots: which entity, what kind of entity, what kind of thing you want back, which relationship connects them, and which way that relationship runs.

Yesterday's experiment measured each slot separately. One was clearly worse than the others, at every difficulty tier:

| tier | entity | entity kind | answer kind | relationship | all five right |
|---|---|---|---|---|---|
| T3 | 0.5098 | 0.5098 | 0.5938 | **0.3651** | 0.2484 |
| T4 | 0.1761 | 0.2201 | 0.4118 | **0.1304** | 0.0818 |
| T5 | 0.2516 | 0.3711 | 0.4706 | **0.2174** | 0.0943 |

EXP-2026-09-014, model arm, instance A, mean of three runs. Tiers are question difficulty: T3 names the entity exactly, T4 paraphrases its name, T5 only describes it.

The relationship slot also had no excuse for being wrong. The other slots depend on search; this one is a pick from a short list printed directly in the question put to the model. And it was implicated in almost every refusal: of the 85 questions the stage declined to answer, **78** were blocked on the relationship, **31** of them because the model named a list position that did not exist.

**The idea.** The database schema already knows which relationship connects two kinds of thing. Counting the connections actually present in the data:

| source | pairs of thing-kinds | pairs where the relationship is unambiguous |
|---|---|---|
| MITRE CAPEC | 17 | 16 |
| MITRE ATT&CK | 19 | 19 |

Counted from the canonical graph snapshot at build time.

So the model need not be asked at all. The system already refuses to let a model choose the *direction* of a relationship for exactly this reason; the change extends that rule one slot across.

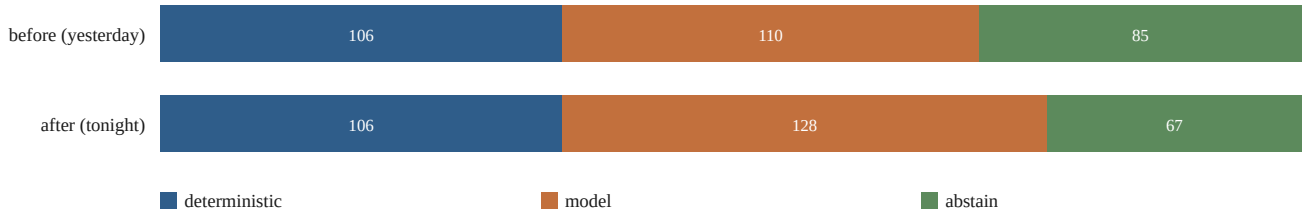
## 2 · What was done, and in what order

The protocol was written and committed **before any measurement**, including the rule for keeping the change and the condition that would count as disproving the idea. That ordering is the only thing that makes a null result worth anything.

1. **Built it.** A schema lookup returning the relationships allowed between two kinds of thing, and a change so the model is consulted only when the pair is ambiguous or undeclared.
2. **Kept an off switch.** So the same model choices could be re-interpreted both ways, making the comparison causal rather than a before-and-after.
3. **Ran two controls** with the feature disabled, to confirm the change is invisible when it is switched off. Both reproduced yesterday's numbers exactly.
4. **Ran the live arms** on both GPU instances in parallel, three times each, clearing the cache between runs so what varies is the model and not the cache.

## 3 · The change works. The number did not move.

Mechanically the intervention does exactly what it was built to do. The relationship is now read from the schema on **116** of the 128 questions the model was consulted on, and the system refuses to answer less often than before.




**How each of the 301 questions got answered, before and after**
Computed from the saved per-question rows of both runs

"Deterministic" means the question was fully understood without any model. "Model" means the model was asked to fill what was left. "Abstain" means the system declined rather than guess. Refusals fall and model-answered questions rise, which is the improvement the change was designed to produce.

And yet the accuracy of that slot is **identical to four decimal places at every single tier** — not close, identical — while the whole-form score is unchanged or slightly worse. The last column is the price paid:

| tier | relationship before | after | change | change in all-five-right | change in confidently wrong |
|---|---|---|---|---|---|
| T1 | 0.2400 | 0.2400 | +0.0000 | +0.0000 | +0.0000 |
| T2 | 0.2400 | 0.2400 | +0.0000 | +0.0000 | +0.0000 |
| T3 | 0.3651 | 0.3651 | +0.0000 | +0.0000 | **+0.0261** |
| T4 | 0.1304 | 0.1304 | +0.0000 | −0.0126 | **+0.0503** |
| T5 | 0.2174 | 0.2174 | +0.0000 | +0.0063 | **+0.0252** |
| T6 | 0.7083 | 0.7083 | +0.0000 | +0.0000 | +0.0000 |

Three-run means on each side, same questions, same model, same settings. Run-to-run spread is at most 0.0225 on any cell above.

The pre-registered condition for calling the idea wrong read: *if the relationship slot rises but the whole-form score does not, the relationship was not the binding constraint.* What happened is weaker still. It did not rise at all. The model had already been picking a relationship the graph agrees with, about as often as the schema does.

### The verdict, against the rule written before the run

| the rule said the change is kept only if… | what happened |
|---|---|
| the relationship slot rises at all three paraphrase tiers | **failed** — identical at all three |
| the whole-form score rises at two of the three | **failed** — identical at two, down at the third |
| confidently-wrong answers do not increase anywhere | **failed** — up at three tiers, both paraphrase tiers among them |

The third clause is the one the protocol singled out in advance as mattering most, and it failed for the reason the protocol predicted. Reading the relationship off two labels turns one kind of error into another: if the system misidentifies *both* what the question is about and what it wants back, the schema will confidently supply the connection between two wrong things. The result is a fluent, specific, wrong answer where before there was a refusal. That is the worst trade this project can make, and it is now measured rather than feared.

**So the change is not kept.** The switches stay off.

### Where the refusals actually come from

The change does reduce refusals, from 85 to 67, and out-of-range picks from 31 to 16. But sorting what is left says plainly why accuracy did not follow:

| what the stage had worked out when it gave up | questions |
|---|---|
| both kinds identified, but the database declares no link between them | 24 |
| never worked out what the question is *about* | 26 |
| knew what it was about, not what was wanted back | 10 |
| blocked on something other than the relationship | 7 |

Only the first row is a relationship problem, and it is one the change handles correctly by refusing rather than guessing. The other 36 are the system not knowing *what kind of thing* the question concerns. That is the label slots, and it is where the next attempt belongs.

## 4 · The finding: the answer key names connections the database does not have

Chasing why the number would not move produced something more useful than the change itself.

The slot scores are graded against a hand-built answer key, written when the question bank was built, that records what each question *means*. Checking that key against the database it is supposed to describe:

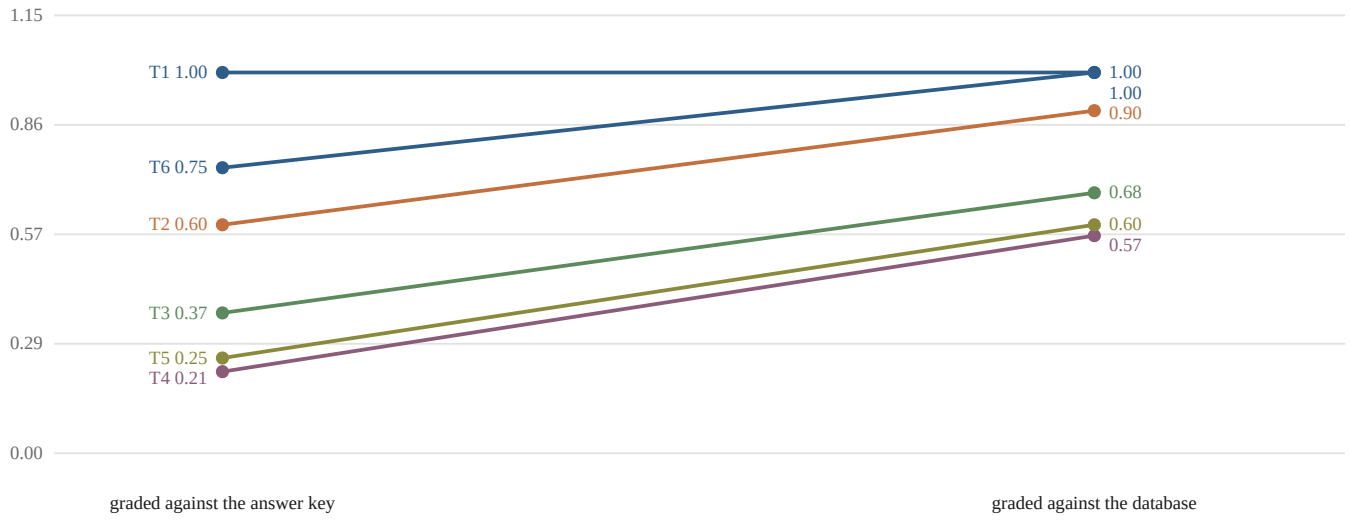
| question is about | asking for | answer key says the link is | the database actually has | questions |
|---|---|---|---|---|
| Group | Technique | `HAS_TECHNIQUE` | `USES` | 4 |
| Campaign | Technique | `HAS_TECHNIQUE` | `USES` | 3 |
| Software | SubTechnique | `HAS_TECHNIQUE` | `USES` | 3 |
| DataComponent | Analytic | `HAS_ANALYTIC` | `MONITORS` | 1 |

All 11 of the 36 answer-key records that name a link between two kinds of thing, checked against the canonical graph snapshot.

### Read that table slowly

`HAS_TECHNIQUE` is a real MITRE ATT&CK connection. But in this data it runs only from a *tactic* to a technique. It does not exist between a *group* and a technique. The connection that does exist there is `USES` — an attacker group *uses* a technique.

So when the system answered `USES`, which is the only answer the database can give, it was scored as wrong. Eleven questions, roughly a third of everything that slot was graded on.




**The same answers, scored twice**
Computed from scripts/rescore_slot_relation.py over the saved rows; the answer key is not edited

Pooled across every graded question, the relationship slot reads **0.4286** against the answer key and **0.7273** against the database. That gap is not the system improving. It is the measurement being wrong, and it had been wrong in yesterday's report too.

### The same defect has a second surface

Fourteen more answer-key entries describe CAPEC questions using ATT&CK vocabulary — asking for a "Technique" from a database that has no such thing and answers through its own cross-reference table instead. Same root cause: the answer key is written in the language people use to *talk* about the data, and graded against the language the data is actually *stored* in.

**Why this was not caught earlier**

It nearly was. The code that turns an answer-key record into a question already detects this case, refuses to build a question from it, and its comment names the cause: 39 of the 120 recorded connections were guessed from the name of the template that generated the question, and a template called `group_techniques` yields `HAS_TECHNIQUE`. That protection sits one layer above the scorer, and the scorer reads the raw record.

**Nothing was repaired.** Editing an answer key after seeing a score is precisely the move that makes evaluations meaningless, so both readings are reported side by side and the choice is filed as a decision for the morning.

## 5 · A second measurement defect, found while setting up the cost run

The second job of the night was to put a number on something the project has claimed for months without measuring: that the deterministic path is not merely more accurate than letting the AI write the queries, but far cheaper.

Run that AI-writes-the-query path the obvious way and it finishes four questions in **2.8 seconds**, returns nothing on all four, and never contacts the model once. It does not crash. It writes a complete, plausible-looking answer for every question:

```
"The attack specialist failed and contributed no evidence:
 pre-run gate rejected all drafts: empty query"
```

That reads exactly like a model with nothing to say. It is a model that was never asked.

**The cause.** The local model reasons at length before answering. At the default output allowance it spends the entire allowance thinking and returns nothing, and the safety gate correctly rejects an empty query. One script in the repository already carries the settings that avoid this; nothing else applies them.

| setting | plain run | the settled configuration |
|---|---|---|
| output allowance | 2,048 tokens | 32,768 tokens |
| patience per call | 30 seconds | 4,096 seconds |
| reasoning steps allowed | 1 | 5 |
| inherited thinking budget | whatever the environment holds | explicitly cleared |

> ### Why this one matters more than it looks
>
> This is the failure that produces a *plausible wrong number* rather than an error. An arm configured this way finishes fast, writes a full set of answers, and declines politely 284 times. Graded, it would have reported the AI-writes-the-query path as **instant and free** — the exact opposite of the truth, and a spectacular false confirmation of the very claim the run was meant to test.
>
> It was caught because the token counter added at the start of the night read zero calls. Without that instrument the run would have produced a clean, confident, entirely wrong cost table.

The runner itself was deliberately **not** changed. Making it refuse to start when a reasoning model meets a small allowance is the right fix, and it is a change in behaviour, so it does not get slipped in alongside a measurement.

## 6 · What each way of answering costs

Two numbers are reported separately, because collapsing them is how a cost claim becomes unfalsifiable. **Throughput** is what it costs to run the whole evaluation at full parallelism. **Single-user latency** is what one analyst waits for one question. A throughput figure divided by a question count is not the second number.

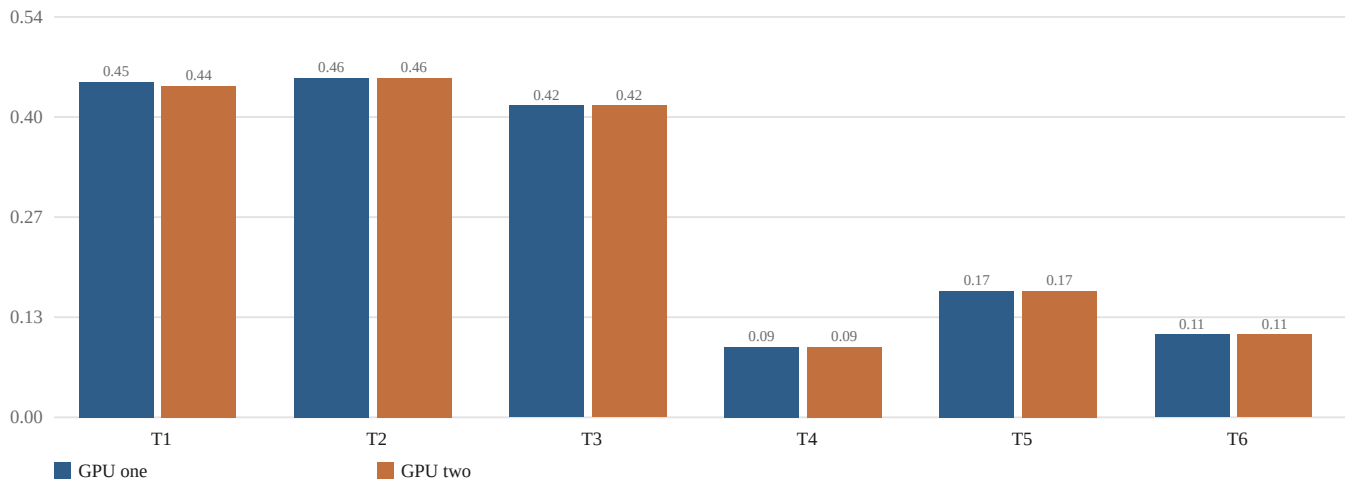
*Throughput run not yet complete.*

*Latency run not yet complete.*

No dollar figure is given. The models are served on hardware this project does not pay for per token, so seconds and tokens are the honest units and a dollar amount would be invented.

## 7 · Does the result repeat?

The same arm was run 3 times on one GPU and 3 times on a second, with the cache cleared between runs so that what varies is the model and not a stored answer. Both instances serve identical weights, so this measures repeatability, not a comparison between models.




**Questions answered with exactly the right records, by difficulty tier**
Computed from 3 runs on one instance and 3 on the other

| tier | GPU one mean | spread | GPU two mean | spread |
|---|---|---|---|---|
| T1 | 0.4497 | 0.0075 | 0.4444 | 0.0000 |
| T2 | 0.4550 | 0.0075 | 0.4550 | 0.0075 |
| T3 | 0.4183 | 0.0092 | 0.4183 | 0.0185 |
| T4 | 0.0943 | 0.0000 | 0.0943 | 0.0000 |
| T5 | 0.1698 | 0.0000 | 0.1698 | 0.0000 |
| T6 | 0.1111 | 0.0000 | 0.1111 | 0.0000 |

## 7b · A third finding, carried in from yesterday and now verified

Earlier in the same working session, before tonight's runs, a diagnostic of the schema-reading path turned up something that had never been written down. It was recomputed from source tonight rather than trusted, and both figures reproduced exactly.

The CAPEC database's spec sheet lists 21 kinds of connection and gives safety rules for 16. The safety gate treats a connection missing from a partially-filled rule set as *forbidden*, and throws the query away before it ever runs:

```
pre-run gate rejected all drafts:
relationship 'HAS_ALTERNATE_TERM' has no allowed_edge_patterns in card
```

The system builds the correct query and is not allowed to ask it.

| | on the 34 questions needing those five connections | on the other 250 |
|---|---|---|
| reads the database's own structure | **0.0000** | 0.6649 |
| hand-written queries | 0.2076 | 0.5452 |

Development half, 284 questions, recomputed 2026-09-10 from the post-repair scores.

An absolute zero, on the approach that leads everywhere else. And the asymmetry is worse than it looks: the rule exists only for one of the two databases, and the *other* database's spec sheet lists no safety rules at all — so it is unconstrained. A spec sheet that is 16-of-21 complete is punished where one that is 0-of-21 is not.

All five missing entries have exactly one possible answer in the data, so the repair is derivable rather than something anyone has to author. It would move the schema-reading path's development score from 0.585 to somewhere between 0.610 and 0.665.

> **Why this is not simply a bug to fix quietly**
>
> An earlier experiment repaired six defects in the **hand-written** approach before comparing the two, precisely so the comparison would be fair. This is the mirror-image defect, and repairing it widens a gap the project already reports in the schema-reading path's favour. It has to be published as that, not slipped in as a bug fix, or the comparison stops being trustworthy.

## 8 · What this means for the October pitch

Nothing in the deck has to change, and one thing in it gets stronger.

### Do not change

The headline comparison between the three ways of answering is untouched. Tonight's work sits entirely inside a stage that ships switched off.

### Gets stronger

The slide asking for the next round of funding currently says: *ask by name and it works, paraphrase the name and it scores zero — fund us to fix that.* That is a gap, not a plan. It can now say something better: we identified the weakest component, built the fix, measured it against a pre-registered rule, and the fix did not work — but the measurement told us why, and it was not what anyone expected. That is what a funded research programme looks like from the inside, and it is far more convincing to an engineer than another round of numbers that only go up.

### The thing to be careful about

Both defects found tonight were **measurement** defects, not system defects, and both would have produced confident, plausible, wrong numbers. If anyone in that room reads this repository, the fact that these were caught and written down — rather than quietly fixed — is the strongest evidence of rigour the project has. It should be said out loud rather than hidden.

## 9 · What was recorded, and what is still open

Everything below is committed. Nothing was pushed.

| record | what it holds |
|---|---|
| `EXP-2026-09-015` | the pre-registered protocol, the null result, and both measurement defects |
| `EXP-2026-09-016` | the cost and latency protocol, the drawn sample, and the silent-empty-query defect |
| `Q44` | the open decision: repair the answer key, or teach the scorer to accept what the database has |
| `scripts/rescore_slot_relation.py` | reports both readings of the slot scores and edits nothing |
| token metering | every model call's cost is now read off the wire rather than estimated |

### The decision waiting for you

The eleven bad answer-key records can be repaired from the database, or the scorer can be taught to accept any connection the database declares between the two kinds of thing the key names. The first is more honest and invalidates two experiments' slot figures; the second is cheaper and leaves a known-wrong answer key in place. Repairing an answer key after seeing a score is exactly the move that evaluation discipline exists to prevent, which is why it was left to you.

### What was not done

The held-back half of the question bank was not touched, and neither was the sealed holdout. No shipped behaviour changed. The flags that control the rewrite stage remain off, as the previous experiment left them.